

Introduction To Machine Learning

Nagesh

May 2026

"I visted the sea in all seasons. It was there all the time , ever restless , ever living , never tired !!!!!"
(Nagesh)

Contents

Notes	6
Some Examples of Distributional Divergence	8
Information Content of an Event	9
Entropy	11
KL Divergence	13
Properties of KL Divergence	16
Minimization of KL Divergence	17
Derivation of Mean Squared Error (MSE) Loss	21
Derivation of Binary Cross-Entropy Loss	24
Derivation of Multiclass Cross-Entropy Loss	25
Notes	28
Molecular Simulations , Human Brain and AI	30
General Formulation	30
Molecular Simulation	31
Deterministic dynamics	31
Stochastic sampling form	31
Equilibrium distribution	31

Human Brain	31
State evolution	31
Action sampling	31
Environment feedback	31
Objective (reward / free energy view)	32
Generative AI	32
Autoregressive models	32
Diffusion models (reverse process)	32
Training objective	32
A Comparison	32
Unified Energy-Based View	33
Interpretations	33
Sampling as Trajectory Generation: A Unified View	33
Molecular Simulation: Sampling Physical Trajectories	33
Example: Particle in a Thermal Bath	33
Trajectory distribution	34
Human Brain: Sampling Behavioral Trajectories	34
Example: Agent in a Room	34
Trajectory distribution	34
Example sampled trajectories	34
Generative AI: Sampling Data Trajectories	34
Example: Language Generation	34
Trajectory distribution	35
Example sampled outputs	35
Unified Perspective	35

Foundational Problem of Machine Learning

The central problem of machine learning can be stated as follows:

Given a dataset

$$D = \{x_i\}_{i=1}^N \stackrel{\text{iid}}{\sim} P_X,$$

where

$$P_X$$

is an unknown probability distribution, estimate the distribution

$$P_X.$$

The fundamental challenge is that the true distribution is completely unknown.

We do not have direct access to:

$$P_X.$$

We only observe samples generated from it:

$$x_1, x_2, \dots, x_N.$$

Any approximation that we construct for the unknown distribution is called a:

Model.

Aside: Estimation Theory

A famous statement in estimation theory is:

“All estimators are wrong, but some are useful.”

The idea is that no model perfectly captures reality.

Instead, models are simplified approximations that are useful for prediction, inference, or generation.

Question

How do we estimate a probability density function from its samples?

General Framework

Suppose we are given:

$$D = \{x_i\}_{i=1}^N \stackrel{\text{iid}}{\sim} P_X.$$

The standard machine learning pipeline consists of three major steps.

(i) Assume a Parametric Functional Form

We first assume a family of distributions parameterized by:

$$\theta.$$

This gives a model:

$$P_\theta.$$

This step is called:

Model Choice / Hypothesis Selection.

The parameters:

$$\theta$$

determine the behavior of the model.

Example 1: Linear Model

Suppose:

$$x \in \mathbb{R}^d.$$

Define:

$$P_\theta(x) = W_1^T x + W_2,$$

where:

$$W_1 \in \mathbb{R}^d, \quad W_2 \in \mathbb{R}.$$

The parameter set is:

$$\theta = \{W_1, W_2\}.$$

This corresponds to a linear parametric model.

Example 2: Gaussian Distribution

Assume:

$$P_\theta(x) = \mathcal{N}(x; \mu, \Sigma),$$

where:

$$\mu \in \mathbb{R}^d, \quad \Sigma \in \mathbb{R}^{d \times d}.$$

The parameters are:

$$\theta = \{\mu, \Sigma\}.$$

Here, the model assumes the data follows a Gaussian distribution.

(ii) Define a Distance Between Distributions

Now we measure how different the model distribution is from the true distribution.

Let:

$$d(P_X, P_\theta)$$

denote a distance (or divergence) between the two distributions.

Formally:

$$d : P_X \times P_\theta \rightarrow \mathbb{R}_{\geq 0}.$$

The objective is:

- small distance $\Rightarrow$ good approximation,
- large distance $\Rightarrow$ poor approximation.

Common choices include:

- Mean Squared Error (MSE),
- Kullback-Leibler divergence,
- Cross-Entropy,
- Wasserstein distance,
- Maximum Likelihood objectives.

(iii) Solve the Optimization Problem

Finally, choose parameters that minimize the distance between the model and the true distribution.

The learning problem becomes:

$$\theta^* = \arg \min_{\theta} d(P_X, P_\theta).$$

This process is called:

Training the Model.

Conceptually:

Data $\implies$ Choose Model $\implies$ Define Distance $\implies$ Optimize Parameters.

Key Insight

Machine learning is fundamentally an approximation problem:

$$\text{Unknown Distribution} \implies \text{Parameterized Approximation.}$$

The role of learning algorithms is to adjust parameters so that the model distribution becomes as close as possible to the unknown data distribution.

Notes

1. **You do not sample from a density function; you sample from a distribution.**

A probability density function:

$$p_X(x)$$

is only a numerical representation of a probability distribution for continuous random variables.

The actual probabilistic object is the distribution:

$$P_X.$$

Sampling means generating realizations:

$$x \sim P_X.$$

The density function merely helps compute probabilities:

$$P(X \in A) = \int_A p_X(x) dx.$$

Thus:

$$\text{Density} \neq \text{distribution.}$$

The density is a representation of the distribution with respect to a measure (typically Lebesgue measure).

Conceptually:

$$\text{Distribution} \implies \text{Sampling}$$

while

$$\text{Density} \implies \text{Probability Computation.}$$

2. P_θ is a design choice.

In machine learning, the true underlying distribution:

$$P_X$$

is completely unknown.

Since directly estimating an arbitrary distribution is impossible in practice, we assume that the unknown distribution belongs to a parameterized family:

$$P_\theta.$$

This assumption is called the:

Model Class / Hypothesis Class.

Thus, we make the modeling assumption:

$$P_X \approx P_\theta.$$

The learning algorithm then searches for parameters:

$$\theta$$

such that:

$$P_\theta$$

approximates the unknown distribution as closely as possible.

3. **Universal Function Approximation**

Some function classes are known to be extremely expressive.

They possess the property of being:

Universal Function Approximators.

This means that, under suitable conditions, these function classes can approximate arbitrary functions to any desired accuracy.

Examples include:

- neural networks,
- transformers,
- kernel methods,
- polynomial expansions.

Conceptually:

$$\forall f, \quad \exists \theta \quad \text{such that} \quad P_\theta \approx f.$$

provided the model has sufficient capacity.

Example: Neural Networks

Consider a neural network:

$$f_\theta(x).$$

The Universal Approximation Theorem states that sufficiently large neural networks can approximate a very broad class of continuous functions arbitrarily well.

Thus, instead of manually deriving complicated functions, machine learning learns them from data by optimizing parameters.

4. Learning is parameter search in a function space.

Once a functional family:

$$P_\theta$$

is chosen, training reduces to searching for parameters:

$$\theta^*$$

such that:

$$P_{\theta^*} \approx P_X.$$

Therefore, modern machine learning can be interpreted as:

Function Approximation + Optimization + Probability Estimation.

Some Examples of Distributional Divergence

Objective

Given two probability distributions defined on the same sample space, the goal is to define a notion of:

“distance” or “difference” between distributions.

Formally, given:

$$P_X, \quad P_Y,$$

defined on the same measurable space, we seek a functional:

$$d(P_X, P_Y)$$

that quantifies how different the two distributions are.

One important approach comes from:

Information Theory.

Information Content of an Event

Consider an event:

$$A \in \mathcal{F}.$$

We want to define a quantity:

$$I(A),$$

that measures how much “information” is conveyed when the event occurs.

Intuition:

- Rare events are more surprising,
- Common events are less surprising.

Thus:

Lower probability $\implies$ Higher information.

Higher probability $\implies$ Lower information.

Desired Properties

The information metric should satisfy:

1. More likely events should convey less information,
2. The entire sample space should convey zero information,
3. Impossible events should convey maximum information.

Definition

A natural definition satisfying these properties is:

$$I(A) = -\log P(A).$$

This quantity is called:

- self-information,
- surprisal,
- information content.

Why Logarithm?

The logarithm converts multiplication of probabilities into addition.

For independent events:

$$P(A_1 \cap A_2) = P(A_1)P(A_2).$$

Therefore:

$$I(A_1 \cap A_2) = -\log(P(A_1)P(A_2)).$$

$$= -\log P(A_1) - \log P(A_2).$$

Hence:

$$I(A_1 \cap A_2) = I(A_1) + I(A_2).$$

This additive property is extremely important in information theory.

Examples

Example 1: Certain Event

If:

$$A = \Omega,$$

then:

$$P(A) = 1.$$

Hence:

$$I(A) = -\log(1) = 0.$$

Interpretation:

“Something in the sample space will happen” conveys no information.

Example 2: Impossible Event

If:

$$P(A) = 0,$$

then:

$$I(A) = -\log(0) = \infty.$$

Interpretation:

Impossible events are maximally surprising.

Example 3: Coin Toss

Suppose a fair coin:

$$P(\text{Head}) = \frac{1}{2}.$$

Then:

$$I(\text{Head}) = -\log\left(\frac{1}{2}\right).$$

Using base-2 logarithm:

$$I(\text{Head}) = 1 \text{ bit.}$$

Thus observing one fair coin toss gives one bit of information.

Now suppose:

$$P(A) = 0.99.$$

Then:

$$I(A) = -\log(0.99),$$

which is very small.

Highly predictable events carry very little information.

Entropy

Suppose:

$$X$$

is a discrete random variable with distribution:

$$P_X(x_i).$$

Each outcome:

$$x_i$$

has associated surprisal:

$$-\log P_X(x_i).$$

Some outcomes are highly surprising, others are predictable.

The average information content of the random variable is obtained by taking expectation under the distribution itself.

This gives:

$$H(P_X) = \mathbb{E}_{P_X}[-\log P_X(X)].$$

For discrete random variables:

$$H(P_X) = -\sum_i P_X(x_i) \log P_X(x_i).$$

This quantity is called:

Entropy.

Interpretation of Entropy

Entropy measures:

- uncertainty,
- randomness,
- average surprisal,
- average information content.

Higher entropy:

$\Rightarrow$

more uncertainty.

Lower entropy:

$\Rightarrow$

more predictability.

Example: Fair Coin

For a fair coin:

$$P(H) = P(T) = \frac{1}{2}.$$

Entropy:

$$H(X) = -\frac{1}{2} \log \frac{1}{2} - \frac{1}{2} \log \frac{1}{2}.$$

Using base-2 logarithm:

$$H(X) = 1 \text{ bit.}$$

This is the maximum possible entropy for a binary variable.

Example: Biased Coin

Suppose:

$$P(H) = 0.99, \quad P(T) = 0.01.$$

Then entropy becomes very small.

Interpretation:

The outcome is highly predictable, therefore it conveys little information on average.

Key Insight

Entropy quantifies the average uncertainty of a probability distribution.

It forms the foundation for many machine learning objectives, including:

- cross-entropy loss,
- KL divergence,
- maximum likelihood estimation,
- variational inference,
- generative modeling.

KL Divergence

Suppose:

$$P_X \quad \text{and} \quad Q_X$$

are two probability distributions defined on the same sample space.

We now ask the following question:

If the true distribution is P_X , but we use Q_X instead, how much extra information or error do we incur?

This idea leads to the notion of:

Kullback-Leibler (KL) Divergence.

Cross Entropy

Recall that entropy of P_X is:

$$H(P_X) = \mathbb{E}_{P_X}[-\log P_X(X)].$$

This measures the average information content of the distribution itself.

Now instead of evaluating surprisal using P_X , suppose we evaluate it using another distribution Q_X :

$$H(P_X, Q_X) = \mathbb{E}_{P_X}[-\log Q_X(X)].$$

For discrete distributions:

$$H(P_X, Q_X) = - \sum_x P_X(x) \log Q_X(x).$$

This quantity is called:

Cross Entropy.

Interpretation

Cross entropy measures:

Average information required when data is actually generated from P_X , but encoded or modeled using Q_X .

If Q_X approximates P_X well, cross entropy will be small.

If Q_X is very different from P_X , cross entropy becomes large.

From Entropy to KL Divergence

Now consider:

$$-H(P_X) + H(P_X, Q_X).$$

Substituting definitions:

$$= \sum_x P_X(x) \log P_X(x) - \sum_x P_X(x) \log Q_X(x).$$

Combining:

$$= \sum_x P_X(x) \log \left(\frac{P_X(x)}{Q_X(x)} \right).$$

This quantity is called the:

Kullback-Leibler Divergence.

Formally:

$$D_{KL}(P_X \parallel Q_X) = \sum_x P_X(x) \log \left(\frac{P_X(x)}{Q_X(x)} \right).$$

Continuous Version

For continuous distributions:

$$D_{KL}(P_X \parallel Q_X) = \int P_X(x) \log \left(\frac{P_X(x)}{Q_X(x)} \right) dx.$$

Interpretation of KL Divergence

KL divergence quantifies:

- the error made by approximating P_X using Q_X ,
- the extra information required when using Q_X instead of the true distribution P_X ,
- how inefficient Q_X is relative to P_X .

Conceptually:

$$D_{KL}(P_X \parallel Q_X) = \text{Cross Entropy} - \text{Entropy}.$$

Since entropy:

$$H(P_X)$$

is fixed for the true distribution, minimizing KL divergence is equivalent to minimizing cross entropy.

This is why cross-entropy losses are widely used in machine learning.

Example

Suppose the true distribution is:

$$P(H) = 0.5, \quad P(T) = 0.5.$$

But suppose our model predicts:

$$Q(H) = 0.9, \quad Q(T) = 0.1.$$

Then:

$$D_{KL}(P \parallel Q) = 0.5 \log \frac{0.5}{0.9} + 0.5 \log \frac{0.5}{0.1}.$$

This value becomes positive because Q poorly approximates the true distribution.

If:

$$Q = P,$$

then:

$$D_{KL}(P \parallel Q) = 0.$$

No extra information is required when the model perfectly matches the true distribution.

Properties of KL Divergence

1. KL divergence is non-symmetric

$$D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P).$$

Thus, KL divergence is not a true mathematical metric.

The direction matters.

Intuitively:

“Approximating P using Q ”

is not the same as

“Approximating Q using P ”.

2. KL divergence is always non-negative

$$D_{KL}(P \parallel Q) \geq 0.$$

This follows from Gibbs’ inequality.

KL divergence can never be negative.

3. KL divergence is zero iff the distributions are identical

$$D_{KL}(P \parallel Q) = 0 \iff P = Q.$$

Thus, zero divergence means perfect distributional agreement.

Importance in Machine Learning

KL divergence is fundamental in modern machine learning.

It appears in:

- maximum likelihood estimation,
- variational autoencoders (VAEs),
- diffusion models,
- reinforcement learning,
- probabilistic inference,
- language model training.

A large fraction of machine learning can be interpreted as minimizing some divergence between:

True Data Distribution and Model Distribution.

Minimization of KL Divergence

Suppose we are given a dataset:

$$D = \{x_1, x_2, \dots, x_N\} \stackrel{\text{iid}}{\sim} P_X,$$

where:

$$x_i \in \mathbb{R}^d.$$

The objective is:

Estimate the unknown data distribution P_X using the observed samples.

Model Assumption

Assume a parametric model density:

$$P_\theta(X).$$

The parameter set:

$$\theta$$

controls the behavior of the model.

Our goal is to choose parameters such that:

$$P_\theta \approx P_X.$$

KL Divergence Objective

A natural way to compare distributions is through KL divergence:

$$D_{KL}(P_X \parallel P_\theta).$$

For continuous distributions:

$$D_{KL}(P_X \parallel P_\theta) = \int P_X(x) \log \left(\frac{P_X(x)}{P_\theta(x)} \right) dx.$$

The learning problem becomes:

$$\theta^* = \arg \min_{\theta} D_{KL}(P_X \parallel P_\theta).$$

Expanding the KL Divergence

Expand the logarithm:

$$D_{KL}(P_X \parallel P_\theta) = \int P_X(x) \log P_X(x) dx - \int P_X(x) \log P_\theta(x) dx.$$

Rewriting:

$$= -H(P_X) - \int P_X(x) \log P_\theta(x) dx.$$

where:

$$H(P_X) = - \int P_X(x) \log P_X(x) dx$$

is the entropy of the data distribution.

Important Observation

The entropy term:

$$H(P_X)$$

depends only on the true data distribution and is completely independent of the model parameters:

$$\theta.$$

Therefore, during optimization:

$$-H(P_X)$$

acts as a constant.

Interpretation:

The intrinsic uncertainty of the data cannot be changed by the model.

The dataset is fixed, hence its entropy is fixed.
We can only optimize the second term.

Final Optimization Objective

Thus:

$$\theta^* = \arg \min_{\theta} - \int P_X(x) \log P_{\theta}(x) dx.$$

Using expectation notation:

$$\theta^* = \arg \min_{\theta} -\mathbb{E}_{P_X}[\log P_{\theta}(X)].$$

This objective is called:

Negative Log-Likelihood (NLL).

Equivalently:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{P_X}[\log P_{\theta}(X)].$$

This is the principle of:

Maximum Likelihood Estimation (MLE).

Approximating Expectations Using Samples

The true expectation:

$$\mathbb{E}_{P_X}[\log P_{\theta}(X)]$$

cannot be computed directly because:

$$P_X$$

is unknown.

However, we have IID samples:

$$x_1, x_2, \dots, x_N.$$

Using the Law of Large Numbers:

$$\frac{1}{N} \sum_{i=1}^N f(x_i) \rightarrow \mathbb{E}_{P_X}[f(X)] \quad \text{as } N \rightarrow \infty.$$

Choosing:

$$f(X) = \log P_\theta(X),$$

we obtain:

$$\frac{1}{N} \sum_{i=1}^N \log P_\theta(x_i) \rightarrow \mathbb{E}_{P_X}[\log P_\theta(X)].$$

Therefore:

$$\theta^* \approx \arg \max_{\theta} \frac{1}{N} \sum_{i=1}^N \log P_\theta(x_i).$$

Equivalently:

$$\theta^* \approx \arg \min_{\theta} -\frac{1}{N} \sum_{i=1}^N \log P_\theta(x_i).$$

Final Interpretation

Thus, machine learning training reduces to:

1. assume a model family:

$$P_\theta,$$

2. evaluate how likely the observed samples are under the model,
3. adjust parameters to maximize likelihood.

Conceptually:

Observed Data $\implies$ Likelihood under Model $\implies$ Optimize Parameters.

Key Insight

Maximum likelihood estimation is equivalent to minimizing KL divergence between:

True Data Distribution and Model Distribution.

Therefore, a large fraction of machine learning can be interpreted as:

Distribution Matching via Optimization.

Derivation of Mean Squared Error (MSE) Loss

Suppose we are given supervised data:

$$D = \{(x_i, y_i)\}_{i=1}^N \stackrel{\text{iid}}{\sim} P_{XY}.$$

Our goal is to estimate the relationship between:

$$X \quad \text{and} \quad Y.$$

Model Assumption

Assume the conditional distribution of Y given X follows a Gaussian distribution:

$$P_{\theta}(Y|X) = \mathcal{N}(Y; h_{\theta}(X), \sigma^2).$$

Interpretation:

- $h_{\theta}(X)$ is the mean of the Gaussian,
- σ^2 is the variance,
- θ are model parameters.

Thus:

$$Y = h_{\theta}(X) + \epsilon,$$

where:

$$\epsilon \sim \mathcal{N}(0, \sigma^2).$$

This means the observed target is assumed to be a noisy version of the prediction function.

Conditional Density

The Gaussian density is:

$$P_{\theta}(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - h_{\theta}(x))^2}{2\sigma^2}\right).$$

Maximum Likelihood Estimation

We now maximize the likelihood of the observed dataset:

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^N P_{\theta}(y_i|x_i).$$

Substituting the Gaussian density:

$$= \arg \max_{\theta} \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - h_{\theta}(x_i))^2}{2\sigma^2}\right).$$

Log-Likelihood

Taking logarithm:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^N \log P_{\theta}(y_i|x_i).$$

This becomes:

$$= \sum_{i=1}^N \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(y_i - h_{\theta}(x_i))^2}{2\sigma^2} \right].$$

Rearranging:

$$= -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - h_{\theta}(x_i))^2.$$

Optimization Objective

The first term:

$$-\frac{N}{2} \log(2\pi\sigma^2)$$

is independent of:

$$\theta.$$

Also:

$$\frac{1}{2\sigma^2}$$

is just a constant scaling factor.

Therefore maximizing likelihood is equivalent to minimizing:

$$\sum_{i=1}^N (y_i - h_{\theta}(x_i))^2.$$

Dividing by N :

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - h_{\theta}(x_i))^2.$$

This is the:

Mean Squared Error (MSE) Loss.

Final Interpretation

Thus, MSE loss naturally emerges from the assumption that:

$$Y|X$$

follows a Gaussian distribution whose mean is predicted by the model:

$$h_{\theta}(X).$$

Conceptually:

Gaussian Noise Assumption $\implies$ Maximum Likelihood $\implies$ MSE Loss.

Important Insight

Different probabilistic assumptions lead to different loss functions.

Examples:

- Gaussian likelihood

$\implies$ MSE Loss

- Bernoulli likelihood

$\implies$ Binary Cross-Entropy

- Categorical likelihood

$\implies$ Cross-Entropy Loss

Thus, many standard machine learning loss functions are simply negative log-likelihoods arising from probabilistic modeling assumptions.

Derivation of Binary Cross-Entropy Loss

Suppose we are given binary classification data:

$$D = \{(x_i, y_i)\}_{i=1}^N,$$

where:

$$y_i \in \{0, 1\}.$$

Our objective is to estimate:

$$P(Y = 1|X = x).$$

Model Assumption

Assume:

$$Y|X \sim \text{Bernoulli}(p_\theta(x)),$$

where:

$$p_\theta(x) = h_\theta(x) \in [0, 1].$$

Interpretation:

$$h_\theta(x)$$

is the probability that the label is 1.

The Bernoulli distribution is:

$$P_\theta(y|x) = (h_\theta(x))^y (1 - h_\theta(x))^{1-y}.$$

This compactly represents both cases:

$$y = 1 \Rightarrow P_\theta(y|x) = h_\theta(x),$$

$$y = 0 \Rightarrow P_\theta(y|x) = 1 - h_\theta(x).$$

Likelihood of Dataset

Assuming IID samples:

$$\mathcal{L}(\theta) = \prod_{i=1}^N P_\theta(y_i|x_i).$$

Substituting Bernoulli likelihood:

$$= \prod_{i=1}^N (h_\theta(x_i))^{y_i} (1 - h_\theta(x_i))^{1-y_i}.$$

Log-Likelihood

Take logarithm:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^N [y_i \log h_{\theta}(x_i) + (1 - y_i) \log(1 - h_{\theta}(x_i))].$$

Optimization Objective

Maximum likelihood estimation gives:

$$\theta^* = \arg \max_{\theta} \log \mathcal{L}(\theta).$$

Equivalently minimize negative log-likelihood:

$$\theta^* = \arg \min_{\theta} - \sum_{i=1}^N [y_i \log h_{\theta}(x_i) + (1 - y_i) \log(1 - h_{\theta}(x_i))].$$

Dividing by N :

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log h_{\theta}(x_i) + (1 - y_i) \log(1 - h_{\theta}(x_i))].$$

This is called:

Binary Cross-Entropy Loss (BCE).

Interpretation

- If the model assigns high probability to the correct class, loss becomes small.
- If the model confidently predicts the wrong class, loss becomes very large.

Thus BCE measures how well predicted probabilities align with true binary labels.

Derivation of Multiclass Cross-Entropy Loss

Now consider multiclass classification.

Suppose:

$$y_i \in \{1, 2, \dots, K\}.$$

The objective is to model:

$$P(Y = k | X = x).$$

Model Assumption

Assume the conditional distribution follows a categorical distribution:

$$Y|X \sim \text{Categorical}(p_1(x), p_2(x), \dots, p_K(x)),$$

where:

$$\sum_{k=1}^K p_k(x) = 1.$$

Typically:

$$p_k(x) = \text{softmax}_k(z(x)).$$

The softmax function is:

$$p_k(x) = \frac{e^{z_k(x)}}{\sum_{j=1}^K e^{z_j(x)}}.$$

One-Hot Representation

Represent labels using one-hot vectors:

$$y_i = (y_{i1}, y_{i2}, \dots, y_{iK}),$$

where:

$$y_{ik} = \begin{cases} 1 & \text{if sample belongs to class } k, \\ 0 & \text{otherwise.} \end{cases}$$

Categorical Likelihood

The likelihood becomes:

$$P_{\theta}(y_i|x_i) = \prod_{k=1}^K p_k(x_i)^{y_{ik}}.$$

Dataset Likelihood

For IID samples:

$$\mathcal{L}(\theta) = \prod_{i=1}^N \prod_{k=1}^K p_k(x_i)^{y_{ik}}.$$

Log-Likelihood

Take logarithm:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log p_k(x_i).$$

Optimization Objective

Minimize negative log-likelihood:

$$\theta^* = \arg \min_{\theta} - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log p_k(x_i).$$

Dividing by N :

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log p_k(x_i).$$

This is the:

Cross-Entropy Loss.

Interpretation

Cross-entropy penalizes assigning low probability to the true class.

If the model predicts:

$$p_k(x_i) \approx 1$$

for the correct class, loss becomes very small.

If the model assigns small probability to the correct class, loss becomes very large.

Connection to KL Divergence

Cross-entropy minimization is equivalent to minimizing KL divergence between:

True Label Distribution and Predicted Distribution.

Thus classification training can fundamentally be interpreted as:

Distribution Matching.

Key Insight

Many common machine learning loss functions arise naturally from probabilistic assumptions and maximum likelihood estimation.

Gaussian Likelihood $\implies$ MSE Loss

Bernoulli Likelihood $\implies$ Binary Cross-Entropy

Categorical Likelihood $\implies$ Cross-Entropy Loss

Notes

1. **Minimum Least Squares Estimation is Maximum Likelihood Estimation under Gaussian assumptions.**

Consider the supervised learning setting:

$$Y = h_{\theta}(X) + \epsilon,$$

where:

$$\epsilon \sim \mathcal{N}(0, \sigma^2).$$

This implies:

$$P(Y|X) = \mathcal{N}(h_{\theta}(X), \sigma^2).$$

Under this probabilistic assumption, maximizing likelihood becomes equivalent to minimizing:

$$\sum_i (y_i - h_{\theta}(x_i))^2.$$

Therefore:

Maximum Likelihood under Gaussian Noise $\iff$ Minimum Mean Squared Error.

Thus, MSE loss is not arbitrary; it directly emerges from assuming a Gaussian conditional density.

2. **Model choice determines the parameterization, and loss choice determines the probabilistic assumption.**

Every machine learning model:

$$h_{\theta}(x)$$

or

$$P_{\theta}(x)$$

is fundamentally a parameterized approximation family.

The parameters:

$$\theta$$

define the hypothesis space that the model can represent.

Simultaneously, every loss function corresponds to an implicit probabilistic assumption about the true data-generating process.

Examples:

Gaussian likelihood $\implies$ MSE loss

Bernoulli likelihood $\implies$ Binary Cross-Entropy

Categorical likelihood $\implies$ Cross-Entropy Loss

Laplace likelihood $\implies$ L1 loss.

Thus:

Model $\implies$ Function Family

while

Loss Function $\implies$ Distributional Assumption.

3. **Most of machine learning can be interpreted as minimizing a divergence metric between distributions.**

The true data distribution:

$$P_X$$

is unknown.

Machine learning constructs a parameterized approximation:

$$P_\theta.$$

Training adjusts parameters so that:

$$P_\theta \approx P_X.$$

This is achieved by minimizing some discrepancy measure:

$$d(P_X, P_\theta).$$

Examples include:

- KL divergence,
- reverse KL divergence,
- Jensen-Shannon divergence,
- Wasserstein distance,
- score matching objectives,
- likelihood-based objectives.

Therefore, modern machine learning can fundamentally be viewed as:

Distribution Estimation + Optimization.

or equivalently:

Learning by Distribution Matching.

Molecular Simulations , Human Brain and AI

All three systems—molecular simulation, the human brain, and generative AI—can be interpreted as stochastic systems that generate samples from an implicit or explicit probability distribution.

General Formulation

A unified abstraction is:

$$x_{t+1} \sim p(x_{t+1} \mid x_t), \tag{1}$$

or in a conditional generative setting:

$$x \sim p_\theta(x \mid c). \tag{2}$$

Each system differs in how p or p_θ is defined and updated.

Molecular Simulation

Molecular dynamics evolves according to physical laws:

Deterministic dynamics

$$m \frac{d^2 x}{dt^2} = -\nabla U(x), \quad (3)$$

where $U(x)$ is the potential energy.

Stochastic sampling form

With thermostats or Langevin dynamics:

$$x_{t+1} \sim p(x_{t+1} \mid x_t; U). \quad (4)$$

Equilibrium distribution

At equilibrium, the system samples:

$$p(x) = \frac{1}{Z} e^{-U(x)/k_B T}. \quad (5)$$

Key property: The distribution is fixed by physics, not learned.

Human Brain

The brain is an adaptive perception–action system.

State evolution

$$h_t = f(h_{t-1}, s_t), \quad (6)$$

where s_t are sensory inputs.

Action sampling

$$a_t \sim \pi(a_t \mid h_t). \quad (7)$$

Environment feedback

$$s_{t+1} = \mathcal{E}(s_t, a_t). \quad (8)$$

Objective (reward / free energy view)

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_t r(s_t, a_t) \right], \quad (9)$$

or equivalently (free energy principle):

$$\pi^* \approx \arg \min_{\pi} \mathcal{F}, \quad (10)$$

where $\mathcal{F}$ is variational free energy.

Key property: The distribution is continuously updated through experience.

Generative AI

Generative models explicitly learn a data distribution.

Autoregressive models

$$p_{\theta}(x) = \prod_t p_{\theta}(x_t \mid x_{<t}). \quad (11)$$

Diffusion models (reverse process)

$$x_{t-1} \sim p_{\theta}(x_{t-1} \mid x_t). \quad (12)$$

Training objective

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)]. \quad (13)$$

Key property: The distribution is learned from data and then sampled.

A Comparison

Aspect	Molecular Simulation	Human Brain	Gen AI
State	Physical coordinates	Neural/belief state	Latent representation
Dynamics source	Physics laws	Biology + learning	Neural network
Distribution origin	Energy $U(x)$	Experience + reward	Data distribution
Sampling	Langevin/MD steps	Policy $\pi(a s)$	Autoregressive/diffusion
Adaptation	None	Continuous learning	Training phase only
Goal	Equilibrium physics	Survival + prediction	Data modeling
Feedback loop	Environment only	Closed-loop agent	Mostly open-loop

Unified Energy-Based View

All three systems can be expressed as sampling from an energy landscape:

$$p(x) \propto e^{-E(x)}. \quad (14)$$

Interpretations

- Molecular simulation:

$$E(x) = U(x) \quad (15)$$

- Human brain (approximate):

$$E(a|s) \approx -(\mathbb{E}[r] - \text{uncertainty}) \quad (16)$$

- Generative AI:

$$E_\theta(x) \sim -\log p_\theta(x) \quad (17)$$

Molecular simulation samples a **physics-defined distribution**, the brain samples an **adaptive reward-conditioned distribution**, and generative AI samples a **learned data distribution**.

Sampling as Trajectory Generation: A Unified View

All three systems—molecular simulation, the human brain, and generative AI—can be understood as **trajectory samplers**. Instead of producing a single deterministic outcome, they generate a random sequence of states:

$$\tau = (x_0, x_1, \dots, x_T), \quad (18)$$

where each trajectory is a sample from a distribution:

$$\tau \sim p(\tau). \quad (19)$$

The key difference lies in how the transition distribution is defined.

Molecular Simulation: Sampling Physical Trajectories

Example: Particle in a Thermal Bath

A particle evolves under thermal fluctuations:

$$x_{t+1} = x_t + v_t \Delta t + \eta_t, \quad (20)$$

where η_t represents thermal noise.

This induces a stochastic transition:

$$x_{t+1} \sim p(x_{t+1} \mid x_t). \quad (21)$$

Trajectory distribution

A full trajectory is:

$$p(\tau) = p(x_0) \prod_{t=0}^{T-1} p(x_{t+1} | x_t). \quad (22)$$

At equilibrium, trajectories follow:

$$p(x) \propto e^{-U(x)/k_B T}. \quad (23)$$

Interpretation: Each simulation run is one sampled microscopic “movie” of atoms consistent with physics.

Human Brain: Sampling Behavioral Trajectories

Example: Agent in a Room

At each time step:

$$a_t \sim \pi(a_t | s_t), \quad (24)$$

and the environment evolves as:

$$s_{t+1} = \mathcal{E}(s_t, a_t). \quad (25)$$

Trajectory distribution

A behavioral trajectory is:

$$p(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi(a_t | s_t) p(s_{t+1} | s_t, a_t). \quad (26)$$

Example sampled trajectories

- τ_1 : look at object $\rightarrow$ walk straight $\rightarrow$ pick up $\rightarrow$ drink
- τ_2 : hesitate $\rightarrow$ explore room $\rightarrow$ sit down

Interpretation: The brain samples one possible *behavioral life path* conditioned on perception and internal goals.

Generative AI: Sampling Data Trajectories

Example: Language Generation

An autoregressive model generates tokens sequentially:

$$x_t \sim p_\theta(x_t | x_{<t}). \quad (27)$$

Trajectory distribution

A generated sequence is:

$$p_{\theta}(x) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}). \quad (28)$$

Example sampled outputs

- “The cat sat on the mat and slept.”
- “The cat sat on the window and stared outside.”
- “The cat sat on the dog and caused chaos.”

Interpretation: Each generation is one sampled trajectory through the space of possible texts.

Unified Perspective

All three systems define a trajectory distribution of the form:

$$p(\tau) = \prod_t p(x_{t+1} \mid x_t), \quad (29)$$

but differ in how the transition kernel is defined:

- **Molecular simulation:**

$$p(x_{t+1} \mid x_t) \text{ from physics (energy + thermal noise)} \quad (30)$$

- **Human brain:**

$$p(a_t \mid s_t) \text{ from learned policy + reward structure} \quad (31)$$

- **Generative AI:**

$$p(x_t \mid x_{<t}) \text{ from learned data distribution} \quad (32)$$

All three systems are **trajectory samplers**: they generate one possible evolution of a system by repeatedly sampling from a learned or physical transition distribution.